UNIVERSITAT POLITÈCNICA DE VALÈNCIA

DISCA
DEPARTAMENTO DE INFORMÁTICA DE SISTEMAS Y COMPUTADORES

Prácticas de la asignatura

Informática Industrial II (GIEIA)

Práctica 0

Toma de contacto con STM32CubeMX y SMT32CubeIDE

Francisco Rodríguez Ballester
prodrig@disca.upv.es

Dep. de Informática de Sistemas y Computadores (DISCA)
Escuela Técnica Superior de Ingeniería Aerospacial y Diseño Industrial (ETSIADI)
Universitat Politècnica de València

Febrero 2026

Prácticas de la asignatura Informática Industrial II (GIEIA)

Práctica 0

Toma de contacto con STM32CubeMX y SMT32CubeIDE

1 Objetivos de aprendizaje **3**

2 Introducción **4**
2.1 Repositorio común `C:\ST\STM32CubeRepository` 4
2.2 Crear el workspace 5

3 Configuración del STM32CubeMX **5**
3.1 Seleccionar el repositorio 5
3.2 Descargar el paquete de firmware para nuestro MCU . . 6

4 Uso del STM32CubeMX **8**
4.1 Seleccionar la tarjeta 8
4.2 Eliminar la configuración de pines y periféricos no deseada 9
4.3 Configurar el proyecto 11

5 Uso del STM32CubeIDE **13**

6 Comprobar que todo funciona **13**
6.1 ST-LINK firmware verification 15

1. Objetivos de aprendizaje

Al final de este documento serás capaz de

1. Gestionar, en el STM32CubeMX, los paquetes de *firmware* necesarios para crear un proyecto adaptado a la tarjeta de prácticas, la STM32F407G-DISC1.

2. Configurar el programa STM32CubeMX para evitar las actualizaciones de dicho *firmware* a fin de que la versión de la librería HAL se mantenga estable durante todo el tiempo que trabajemos con los programas.

3. Crear, también en el STM32CubeMX, un proyecto con la configuración básica del *hardware* de la tarjeta STM32F407G-DISC1 con generación automática del código de soporte de la librería HAL.

4. Editar los ficheros del proyecto en el STM32CubeIDE.

5. Añadir tu propio código usando el STM32CubeIDE a los ficheros del proyecto generados automáticamente por STM32CubeMX.

6. Usar un *workspace* como carpeta raíz de tus proyectos en el STM32CubeIDE.

2. Introducción

Antes de usar los programas STM32CubeMX y STM32CubeIDE es necesario tener cuenta en la página de MyST (`https://my.st.com/cas/login?lang=en`) usando el correo oficial de la UPV. **Si todavía no te has dado de alta, es el momento de hacerlo**.

Actividad 1— Registro en MyST

Si no lo has hecho todavía, regístrate en la página de MyST (`https://my.st.com/cas/login?lang=en`) usando el correo oficial de la UPV.

Tras la instalación de los programas es necesario configurar dónde se ubicará el *repositorio* de los paquetes de firmware: Una carpeta en el disco del ordenador donde se van a almacenar los paquetes de *firmware* necesarios para el funcionamiento del programa.

Esta carpeta, en nuestro ordenador personal (portátil, ordenador de casa), podemos dejarlo a su valor por defecto[1].

Sin embargo, en el laboratorio, y debido a que la primera vez que se hace es un proceso lento, es preferible configurar un repositorio común para todos los usuarios y, además, evitar las actualizaciones a lo largo del curso[2].

De manera que, por un lado, vamos a hacer una instalación *común* del repositorio de paquetes de firmware, para evitar que haya una alta probabilidad de que cada vez nos sentemos en un ordenador distinto del laboratorio tengamos que esperar a la descarga e instalación del paquete de firmware para nuestro microcontrolador, y, por otro lado, vamos a configurar el sistema para que no actualice el paquete de firmware una vez descargado.

2.1. Repositorio común `C:\ST\STM32CubeRepository`

Vamos a utilizar, como carpeta de repositorio de paquetes de firmware, la carpeta `C:\ST\STM32CubeRepository`.

Antes de iniciar el programa STM32CubeMX abre el explorador de archivos y verifica que la carpeta `C:\ST\STM32CubeRepository` existe. Si no es así, créala.

Actividad 2— Repositorio `C:\ST\STM32CubeRepository`

Antes de iniciar el programa STM32CubeMX, abre el explorador de archivos y verifica que la carpeta `C:\ST\STM32CubeRepository` existe. Si no es así, créala.

[1] Generalmente la ubicación por defecto es una subcarpeta `STM32Cube/Repository` dentro de las carpetas personales del usuario.

[2] Para garantizar que una nueva versión del firmware no "rompe" el código de nuestros proyectos. Esto podría ocurrir si la nueva versión no es 100 % compatible con la que venimos utilizando.

2.2. Crear el workspace

El *workspace*[3] es una carpeta donde se van a ubicar los proyectos y aplicaciones embebidas que desarrollemos con estos programas.

Debe ser, por tanto, una carpeta personal y diferenciada para cada usuario, así que deberías elegir una subcarpeta dentro de las carpetas de usuario del ordenador o, mejor aún, una carpeta del disco `W:`, para tener acceso al workspace desde cualquier ordenador del laboratorio y desde tu ordenador personal[4].

MUY IMPORTANTE: Caracteres válidos

Sólo se pueden utilizar caracteres que estén en el alfabeto inglés, espacios en blanco y guiones en la ruta al proyecto; por tanto, esto afecta tanto al nombre del propio proyecto como al de la carpeta workspace y de todas las carpetas que forman la ruta, debiendo descartar las vocales acentuadas, ü, ñ, ç, etc.

La carpeta del workspace debe estar vacía. A lo largo de este documento asumiremos que el workspace es la carpeta `W:\II2\Workspace`. *Ajusta este nombre a la carpeta que hayas elegido.*

Actividad 3— Crear el workspace

Abre el explorador de archivos y crea la carpeta `W:\II2\Workspace`.

3. Configuración del STM32CubeMX

Los pasos descritos en esta sección deben ser llevados a cabo una sola vez si se siguen las instrucciones correctamente.

3.1. Seleccionar el repositorio

Abre el programa STM32CubeMX y selecciona la opción '*Help*' → '*Connection&Updates*' (ver Figura 1) y modifica la configuración:

1. Cambia la carpeta *Repository Folder* a `C:\ST\STM32CubeRepository`.
2. Selecciona *Manual Check* para que el programa no tome acciones por su cuenta. En el futuro, *nunca hagas Check For Updates* a menos que se te indique explícitamente.

[3]En realidad podemos tener varias carpetas que actúen como workspace, pero vamos a simplificar y usar únicamente un workspace.

[4]Incluso desde fuera del campus, si con tu ordenador conectas a la VPN de la UPV, puedes montar la unidad `W:` como una unidad de disco remota.

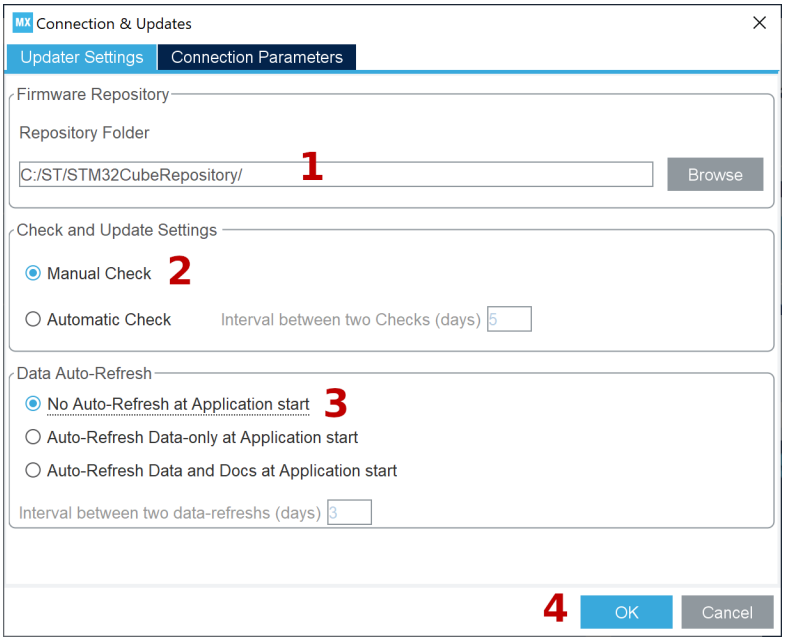

Figura 1: Diálogo '*Help*' → '*Connection&Updates*' de STM32CubeMX.

3. Selecciona *No Auto-Refresh at Application star* para que el programa no busque nuevas versiones del programa ni de los paquetes de firmware instalados.

4. Pulsa el botón *OK*.

3.2. Descargar el paquete de firmware para nuestro MCU

Dentro del programa STM32CubeMX, selecciona la opción '*Help*' → '*Manage embedded software packages*', y busca en la lista de *STM32Cube MCU Packages* la última versión para los **STM32F4** que, en el momento de escribir este documento, es la versión 1.28.3; selecciona dicha entrada de la lista y pulsa el botón *Install* tal y como se muestra en la Figura 2.

Después de un tiempo de espera en el que el programa descarga el paquete de firmware seleccionado (ver Figura 3) nos solicita que aceptemos la licencia, cosa que haremos (ver Figura 4) y, tras unos minutos, el paquete queda descargado e instalado en el ordenador.

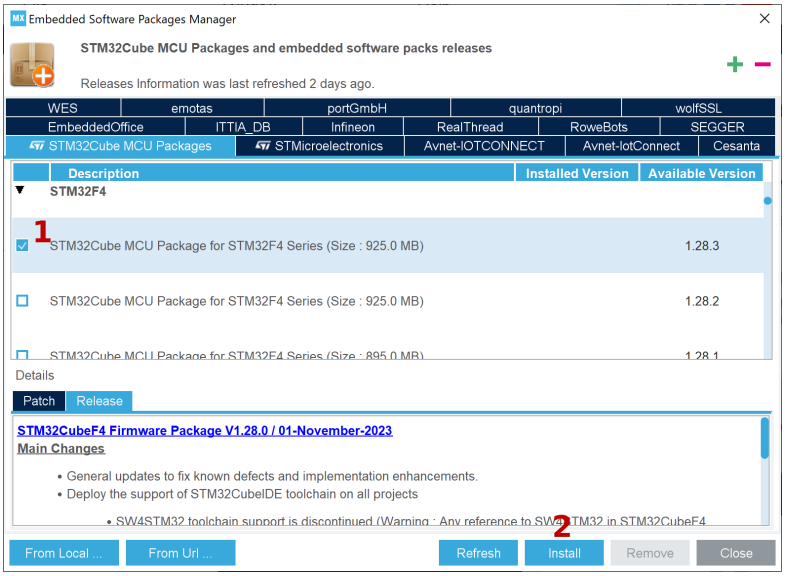

Figura 2: Diálogo '*Help*' → '*Manage embedded software packages*' de STM32CubeMX.

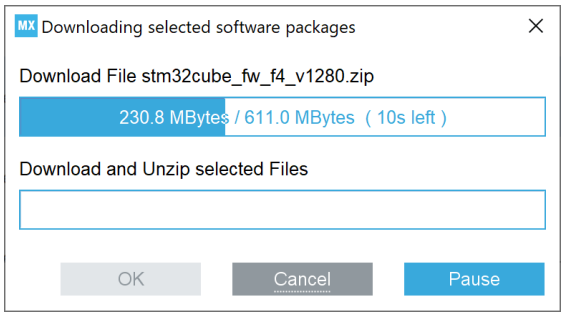

Figura 3: Descarga e instalación de un paquete de firmware.

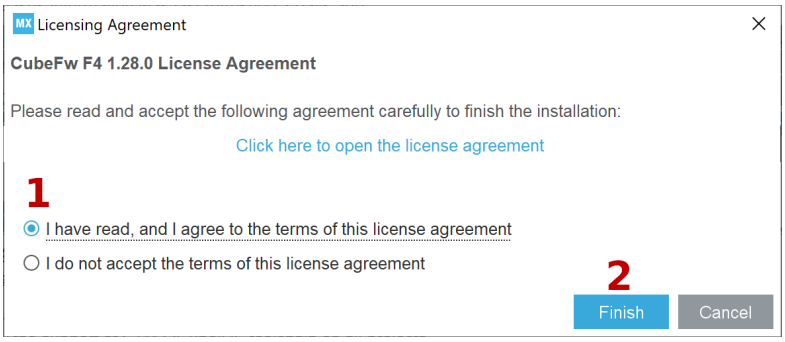

Figura 4: Aceptar licencia de uso del paquete de firmware.

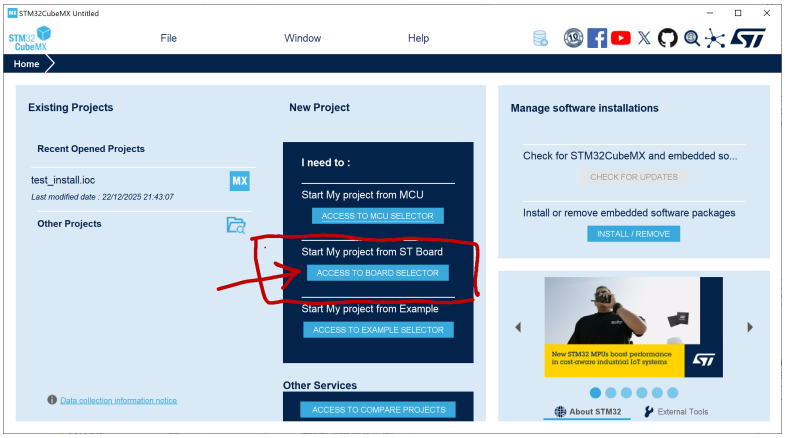

Figura 5: Pantalla principal del STM32CubeMX.

4. Uso del STM32CubeMX

El programa STM32CubeMX sirve para configurar gráficamente el hardware del MCU que queremos utilizar en nuestro proyecto además de para generar una versión inicial del código del mismo que incorpora las funciones necesarias de la librería HAL[5] para gestionar los periféricos que hayamos configurado.

A partir de esta versión inicial del proyecto generada por el STM32CubeMX, usaremos el segundo programa, el STM32CubeIDE, para añadir nuestro propio código, compilar, descargar y depurar el programa.

Siempre podemos volver a usar el STM32CubeMX para modificar la configuración de los pines, periféricos internos o sistem de reloj, por ejemplo; lo que hará el programa es regenerar el proyecto **sobreescribiendo los ficheros .h y .c** del mismo, por lo que es importante que aprendamos a escribir nuestro código de manera que no se pierda si tenemos que usar el STM32CubeMX por segunda vez en un proyecto.

4.1. Seleccionar la tarjeta

La manera más fácil de crear la versión inicial de un proyecto cuando usamos, como es nuestro caso, una tarjeta que está directamente soportada por el software, es usar el boton *Access to board selector* de la pantalla principal, tal y como se muestra en la Figura 5.

En ese momento nos aparece una ventana de selección (Figura 6) donde debemos elegir, de entre las decenas de tarjetas soportadas, la nuestra. Lo más sencillo es usar el filtro *Commercial Part Number* y escribir, al menos parcialmente, el nombre comercial de nuestra tarjeta: **STM32F407-DISC1**; a medida que vamos escribiendo en el filtro se van reduciendo las opciones hasta que hay solamente una tarjeta (la nuestra) o la lista es lo bastante reducida como para localizarla fácilmente.

[5] HAL — *Hardware Abstraction Layer*

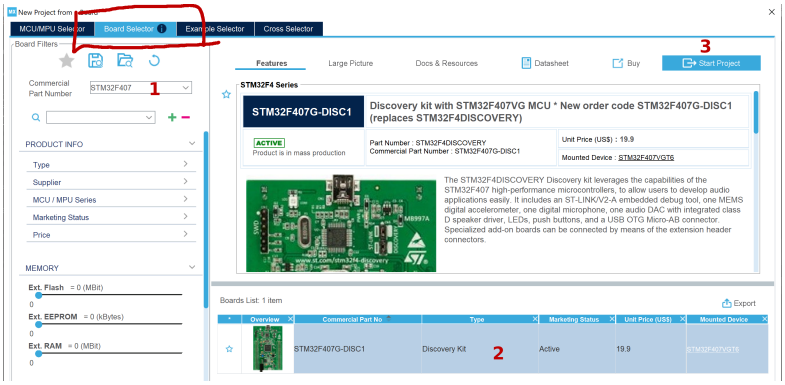

Figura 6: Selección de la tarjeta STM32F407-DISC1.

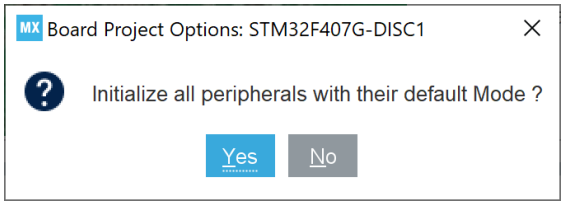

Figura 7: Configuración por defecto de pines y periféricos (*Yes*).

A partir de aquí basta con seleccionar nuestra tarjeta de la lista *Board List* y pulsar el botón *Start Project*.

Lo primero que hace el programa es preguntar si queremos utilizar la configuración por defecto de los pines (Figura 7), a lo que responderemos *Yes*. La enorme ventaja de usar una tarjeta directamente soportada por el programa es ésta: el programa sabe qué elementos hay en la tarjeta y a qué pines del MCU están conectados, de manera que puede configurar directamente todos los pines directamente 'usables' en nuestra tarjeta además de configurar los periféricos internos correspondientes.

4.2. Eliminar la configuración de pines y periféricos no deseada

A pesar de lo cómodo que es que el programa configure correctamente todos los pines del MCU que estén conectados a algún dispositivo de la tarjeta, lo cierto es que en ocasiones no queremos utilizar *todos* los elementos de la misma, sólo unos pocos, por lo que la configuración de dichos elementos 'molesta'.

Es decir, con la inicialización de los periféricos en su modo por defecto el programa automáticamente configura todos los periféricos del microcontrolador necesarios para gestionar los dispositivos que el fabricante sabe que tiene la tarjeta: LEDs, botones, USB, micrófono, etc. Esto nos viene bien para saber lo que hay en la tarjeta, pero añade mucho código que no vamos a usar (por ejemplo, no queremos usar el USB) de manera que vamos a eliminar todo lo innecesario.

Lo que vamos a hacer es eliminar la configuración de esos pines y periféricos que no

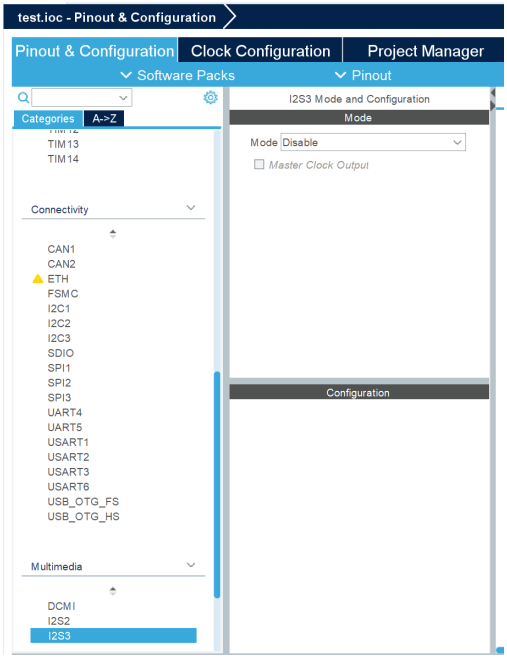

Figura 8: Vista de los periféricos después de cambiar los que no queremos usar a *Disable*.

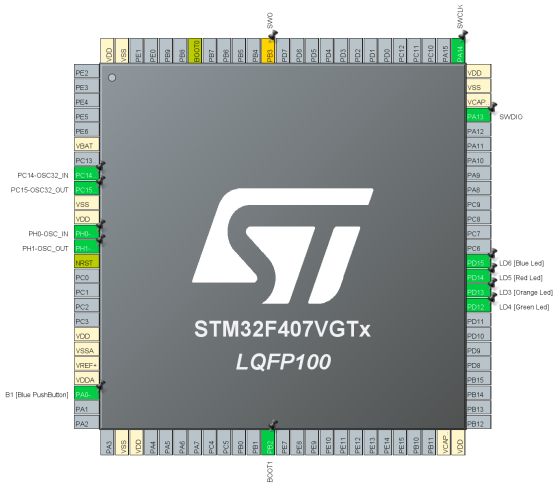

Figura 9: Configuración de pines del microcontrolador después de cambiar los pines que no queremos usar a *Reset_State*.

vamos a utilizar, para quedarnos solamente con lo que sí vamos a usar, empezando por los periféricos internos.

Actividad 4— Eliminar periféricos innecesarios

En la pestaña *Pinout & Configuration*, sección *Connectivity*, selecciona los items en verde y, al aparecer la configuración del mismo, selecciona el modo *Disable*. Esto deberías aplicarlo a los items I2C1, SPI1 y USB_OTG_FS.

Y en la sección *Multimedia*, haz lo mismo con el ítem I2S3.

El resultado debería ser el de la Figura 8.

Y ahora hacemos lo mismo con los pines que no queremos usar.

Actividad 5— Eliminar pines innecesarios

En el *pinout view*, donde aparece la imagen del chip con todos los pines, selecciona uno a uno todos los pines con nombres que comiencen por SPI1_, OTG_, MEMS_, PDM_, Audio_, I2S3_, CS_I2C_ y los pines indicados como GPIO_EXTI10, CLK_IN y VBUS_FS.

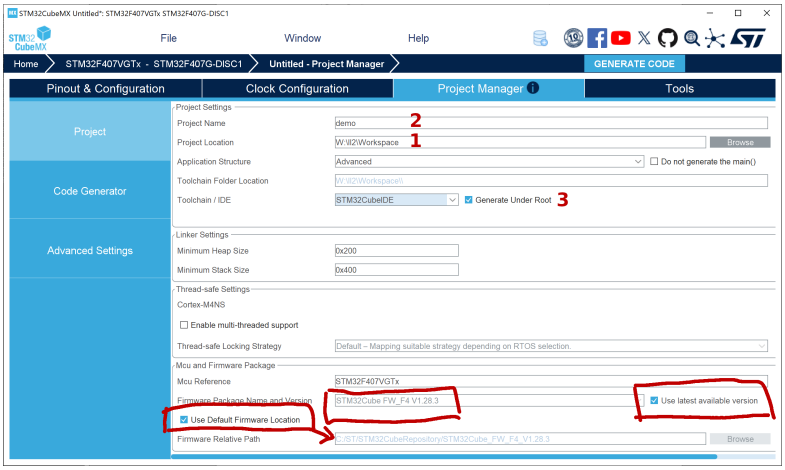

Figura 10: Configuración del proyecto.

Selecciona cada uno de ellos y, en la lista de modos de trabajo disponibles para el pin, selecciona el modo *Reset_State*, que es el primero de la lista y que significa que no vamos a usar dicho pin.

El resultado debería ser el de la Figura 9.

4.3. Configurar el proyecto

Una vez que ya tenemos la configuración del MCU hay que configurar el proyecto antes de generar el código.

Para ello vamos a la pestaña *Project Manager* y, de la columna izquierda, comenzamos seleccionando la sección *Project* (ver Figura 10), donde debemos indicar:

1. *Project Location*. Selecciona la carpeta del workspace (`W:\II2\Workspace`) con el botón *Browse*.

2. Indica el nombre del proyecto en *Project Name*; por ejemplo `demo`.

3. La estructura de la aplicación debe ser *Advanced* y el IDE el STM32CubeIDE.

Fíjate que el campo *Toolchain Folder Location* queda como la unión de la carpeta workspace y el nombre del proyecto. Esto es así porque cada proyecto será una subcarpeta de la carpeta del workspace.

Por lo demás, comprueba que en el apartado *MCU and firmware package* aparece el paquete de firmware que descargaste e instalaste en la Subsección 3.2 con la marca *Latest available version* y que la ruta a dicho paquete está en la carpeta del repositorio común `C:\ST\STM32CubeRepository`.

Pasa entonces a seleccionar de la columna de la izquierda la entrada *Code Generator* donde deben aparecer marcadas las opciones igual en como aparecen en la Figura 11 antes de pulsar el botón *Generate Code* como paso final.

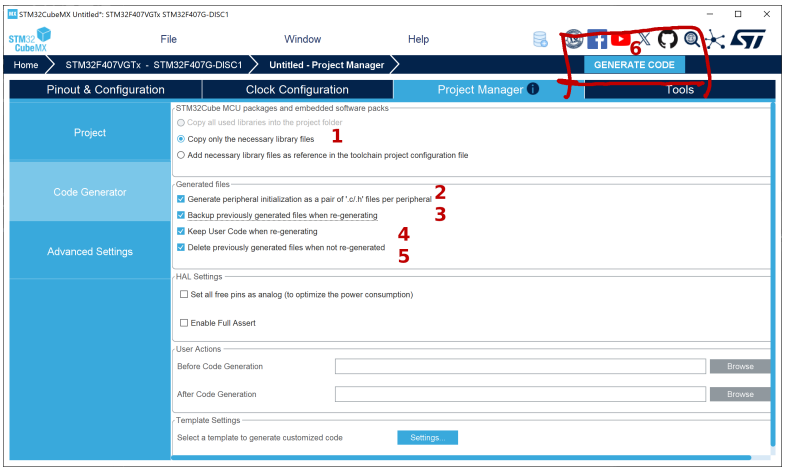

Figura 11: Configuración de la generación de código.

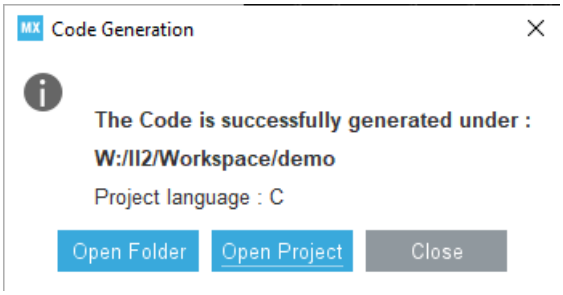

Figura 12: Pregunta tras la generación de código.

Con esto se genera la versión inicial del proyecto y el programa nos permite abrirlo en el segundo programa (ver Figura 12), el STM32CubeIDE, y lo haremos **de dos formas distintas:**

1. Si el programa STM32CubeIDE no está abierto (o está abierto en otro workspace), abriremos el proyecto pulsando el botón *Open Project*, que lanza el STM32CubeIDE e importa el proyecto en el workspace de forma automática.

2. Si, por el contrario, el STM32CubeIDE ya está en ejecución en el mismo workspace donde el STM32CubeMX ha generado el proyecto, entonces **no podemos abrir** una segunda instancia del STM32CubeIDE usando el mismo workspace (el programa está configurado para no permitir esto), por lo que pulsaremos el botón *Close* y, sencillamente, cambiaremos al STM32CubeIDE y, pulsando con el botón derecho del ratón en el proyecto, usaremos la opción *Refresh*.

En cualquier caso ya podemos cerrar el STM32CubeMX y centrarnos en el STM32CubeIDE.

5. Uso del STM32CubeIDE

Si todo ha ideo bien el resultado debería ser similar al de la Figura 13, donde se muestra la lista de ficheros de la subcarpeta `Drivers/STM32F4xx_HAL_Driver`[6], que corresponde a los drivers de aquellos periféricos empleados en el microcontrolador (la carpeta `Inc` corresponde a los ficheros de cabecera, `.h`, y la `Src` a los de código, `.c`) y de la subcarpeta `Core` que corresponde a la aplicación.

De momento la aplicación está vacía de funcionalidad más allá de las funciones añadidas por el STM32CubeIDE para configurar los periféricos, como `MX_GPIO_Init()`, que usa las funciones HAL para configurar los pines que hemos configurado en el *pinout view* del STM32CubeMX como *GPIO_Input* (el botón azul de la tarjeta) o *GPIO_Output* (los LEDs de la tarjeta).

6. Comprobar que todo funciona

Ahora que ya tenemos un proyecto, conecta la tarjeta al ordenador y usa el menú *"Run"*→ *"Debug"* para lanzar la depuración del programa. Como el proyecto no ha sido compilado todavía, esta acción lanza automáticamente la compilación antes de conectar con la tarjeta.

Cuando aparezca la ventana de *launch configuration properties* deja las opciones por defecto y pulsa *OK*.

Es posible que nos aparezca la pregunta sobre el cambio de perspectiva, para pasar a la perspectiva de depuración (*Debug perspective*). Como siempre, marca la opción *Remember my decision* y pulsa *Switch*.

[6] *Hardware Abstraction Layer*. HAL es una librería en código fuente que nos facilita el uso de los periféricos del microcontrolador mediante funciones, tipos de datos, etc. sin tener que acceder directamente a los registros.

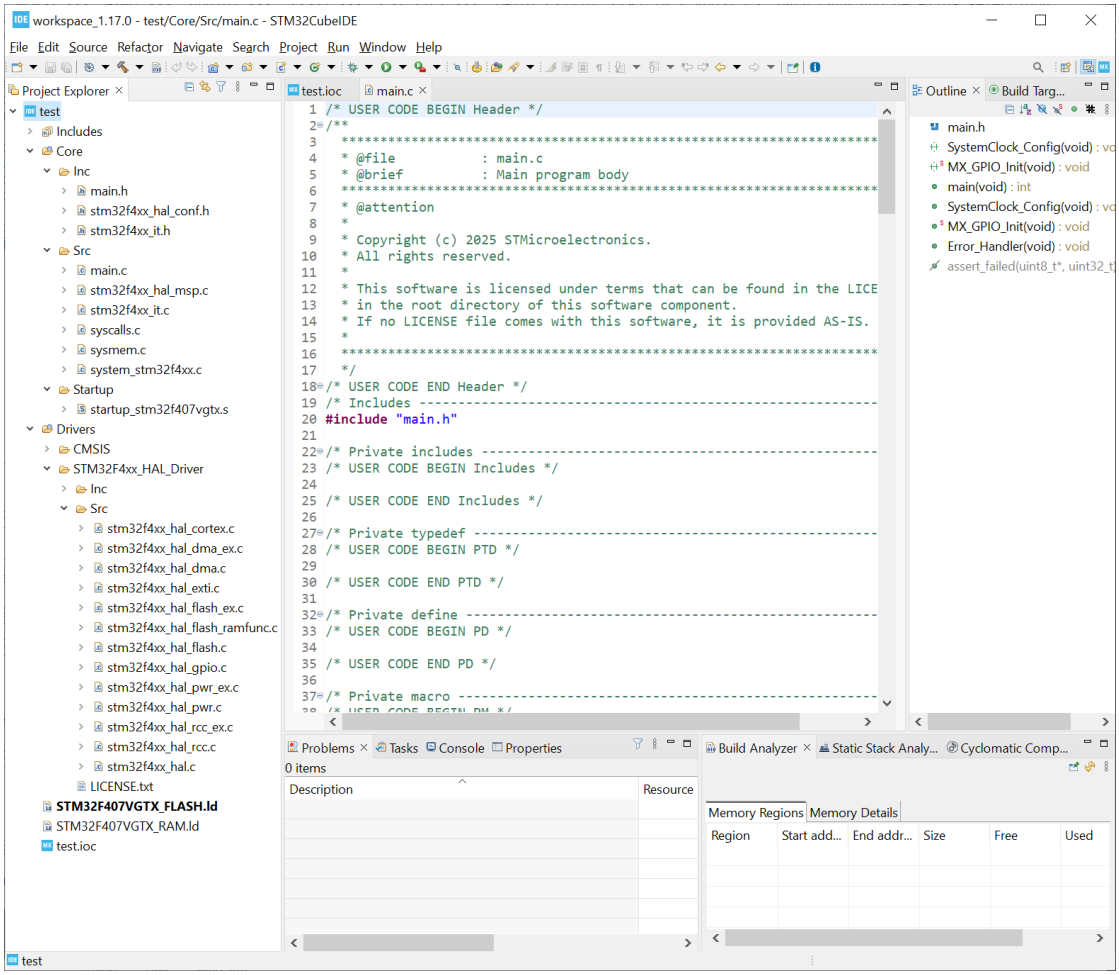

Figura 13: Proyecto test con el código generado.

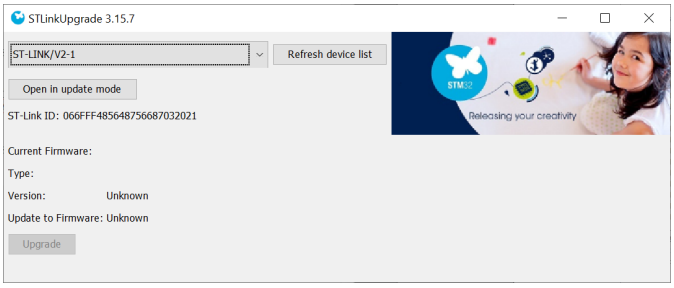

Figura 14: Programa *STLinkUpgrade* para actualizar el firmware del depurador.

Actividad 6— Lanzar la depuración

Usa el menú *"Run"*→ *"Debug"* para lanzar la depuración. Como esto lanza automáticamente la compilación del proyecto, **si aparecen errores o warnings avisa al profesor**.

MUY IMPORTANTE: Si aparece una ventana solicitando permiso del administrador del sistema para acceder a redes avisa al profesor o al técnico de laboratorio, porque sin estos permisos el programa no funciona correctamente.

6.1. ST-LINK firmware verification

Es posible que al iniciar la depuración te aparezca una ventana indicando que hay una nueva versión de firmware para el ST-LINK[7]: Marca la opción *Always check for latest version* y pulsa *Yes*.

Nos aparecerá la ventana del programa *STLinkUpgrade* (ver Figura 14) donde los pasos a seguir son:

1. Hacer click en *Open in update mode*.
2. Esperar a la conexión con la tarjeta.
3. Hacer click en *Upgrade*.
4. Esperar a que se complete el proceso. Al finalizar en la línea de estado del programa se indica *Upgrade successful*.
5. Cerrar el programa.

[7]ST-LINK es el nombre del canal de depuración más popular de los microcontroladores STM.

Actividad 7— Actualizar el firmware de depuración

Si has tenido que actualizar el firmware del depurador con el programa *STLinkUpgrade*, una vez cerrado el *STLinkUpgrade* vuelve al programa STM32CubeIDE y repite el lanzamiento de la depuración con el menú *"Run"*→ *"Debug"*.

Si has tenido éxito, el último mensaje en la consola será *Download verified successfully* y la primera línea de código de la función `main()` estará resaltada en verde y con una pequeña flecha a la izquierda del número de línea (ver Figura 15). Esta línea es la que se va a ejecutar (aún no se ha ejecutado) y es el confirmación de que toda la instalación funciona correctamente.

Actividad 8— Terminar la depuración

Usa el menú *"Run"*→ *"Terminate"* para terminar la depuración.

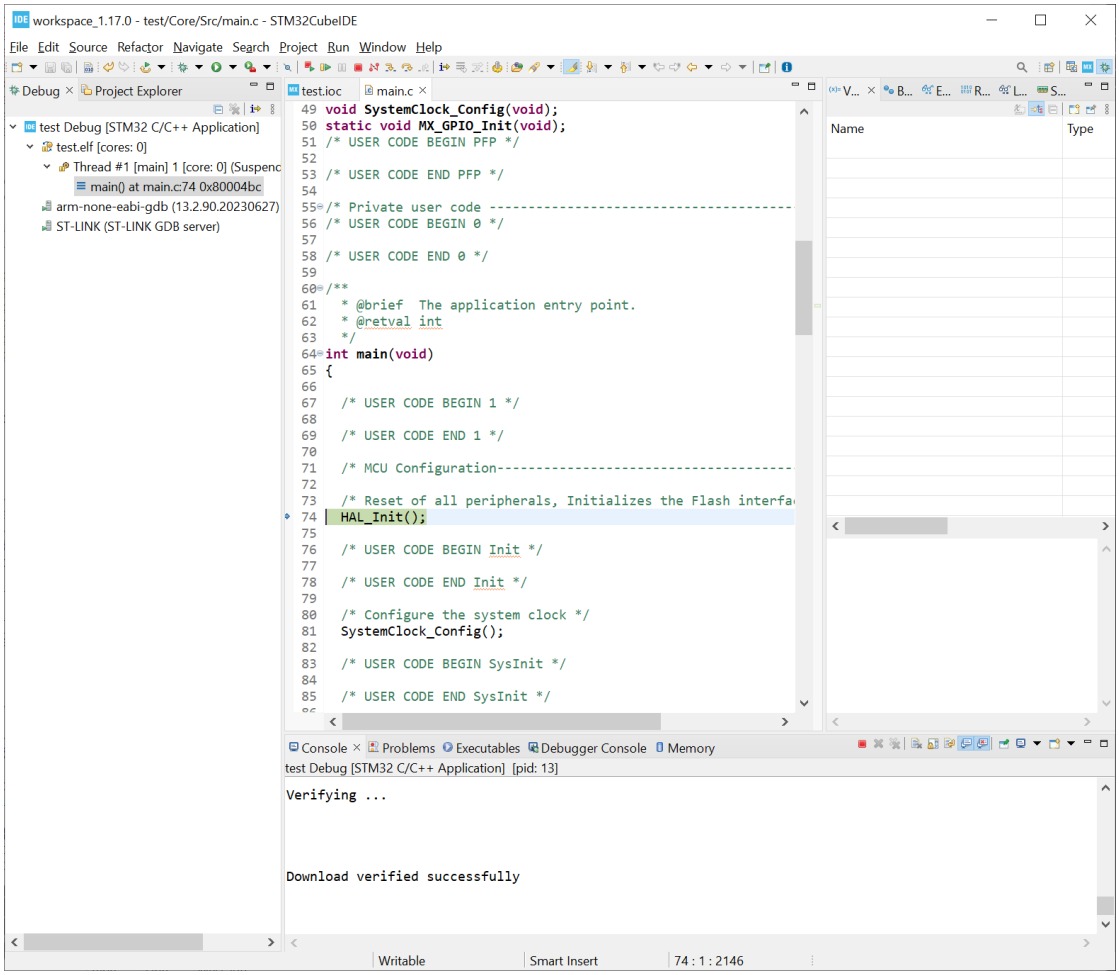

Figura 15: Proyecto test con la depuración iniciada.